Projecte Hibernate i Spring 2026-27

0. CONTROL DE CANVIS (Obligatori)

	Realitzat per	Revisat per	Aprovat per
Nom i cognom (últim canvi)	**Clàudia Reina**		
Càrrec (últim canvi)	Professora		
Data (últim canvi)	2026-09-27		

Llistat de les modificacions (S'inicia i acaba a la mateixa reunió només posar la revisió 0.0 per saber la data de creació del document)		
Revisió núm	**Data**	**Descripció de la modificació**
0.0	2026-09-…	Creació del document
0.1		
0.2		
0.3		
0.4		

★ ***Tan bon punt el document s'imprimeixi o es descarregui perd la seva vigència.***

Els Colons de Catan és un joc de tauler d'estratègia en què els jugadors colonitzen una illa formada per hexàgons que produeixen recursos. Amb aquests recursos construeixen carreteres, poblats i ciutats fins a aconseguir els punts de victòria necessaris. En aquesta pràctica crearem una versió simplificada dels Colons de Catan amb Hibernate i Spring.

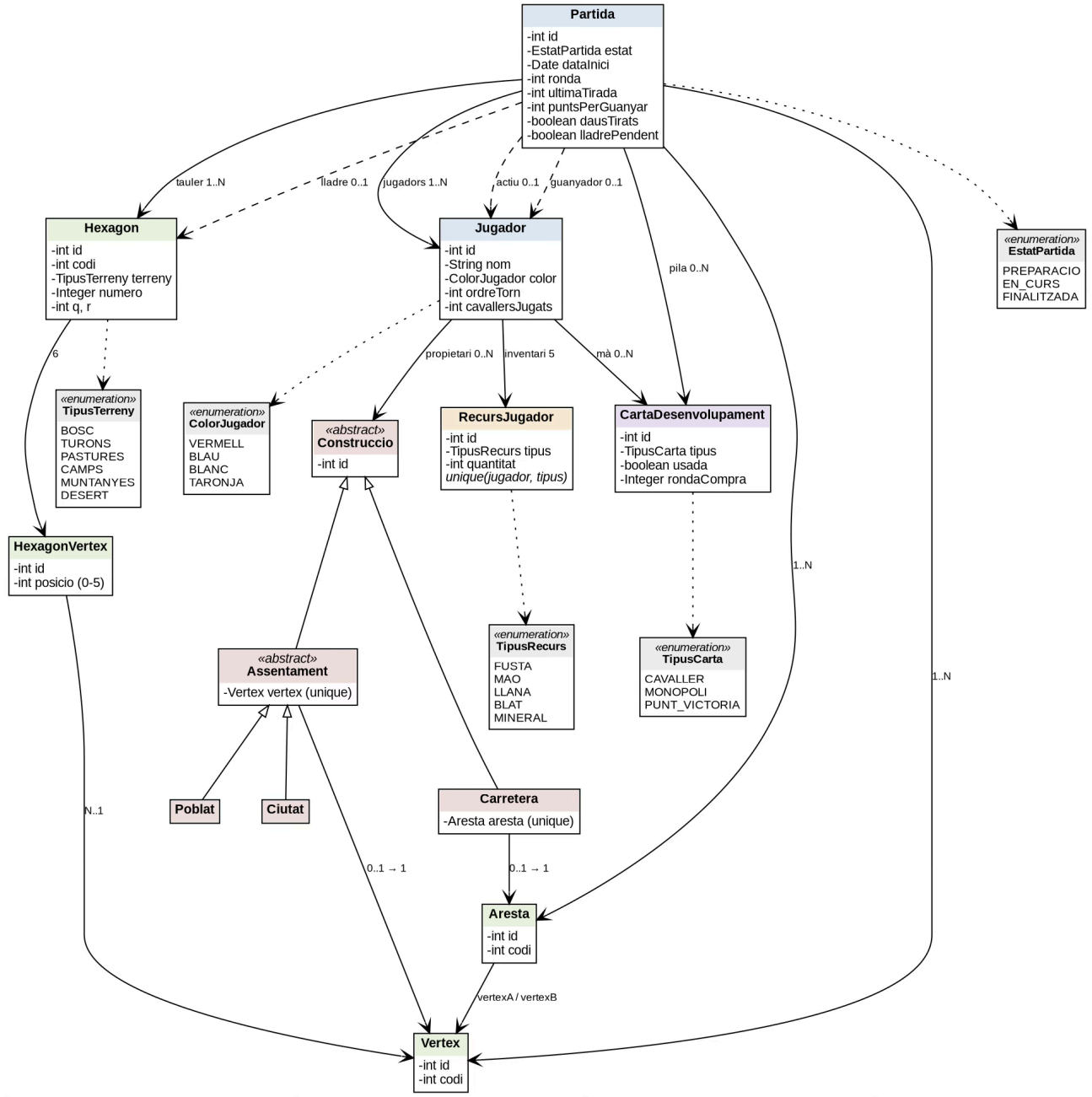
Disseny de la DB

El més important a tenir en compte són el tauler (hexàgons, vèrtexs i arestes) i els recursos de cada jugador. Catan tindrà:

- **Tauler**: 19 hexàgons, 54 vèrtexs i 72 arestes. La topologia (quins vèrtexs formen cada hexàgon i quines arestes uneixen quins vèrtexs) es proporciona en fitxers CSV (vegeu Consideracions). Cada hexàgon, vèrtex i aresta guarda l'id que té al CSV al camp codi.
    - Cada hexàgon té un terreny que produeix un recurs: BOSC → FUSTA, TURONS → MAÓ, PASTURES → LLANA, CAMPS → BLAT, MUNTANYES → MINERAL, DESERT → res.
    - Cada hexàgon (excepte el desert) té un número entre 2 i 12 (mai el 7). Quan surt aquest número als daus, l'hexàgon produeix. El desert no té número (numero és null).
    - Els vèrtexs són els punts on es construeixen poblats i ciutats; les arestes són els segments on es construeixen carreteres. Dos vèrtexs són adjacents si hi ha una aresta que els uneix.
- **Jugador**: Cada jugador té un nom, un color, un ordre de torn i un inventari de recursos. L'inventari es guarda a la taula RecursJugador (jugador, tipus de recurs, quantitat), amb id propi i una restricció d'unicitat sobre (jugador, tipus). En crear el jugador es creen les seves 5 files amb quantitat 0. Els colors possibles (ColorJugador) són VERMELL, BLAU, BLANC i TARONJA.
- Construccions (herència amb estratègia JOINED): Totes hereten de la classe abstracta **Construccio**, que té el propietari. **Poblat** i **Ciutat** hereten d'una classe abstracta intermèdia **Assentament**, que té el vèrtex (únic: un vèrtex només pot tenir un assentament). **Carretera** hereta directament de Construccio.
- **Poblat**: es construeix sobre un vèrtex. Val 1 punt i produeix 1 recurs.
- **Ciutat**: substitueix un poblat propi. Val 2 punts i produeix 2 recursos.
- **Carretera**: es construeix sobre una aresta. Serveix per connectar nous poblats.
- **Lladre**: La partida guarda a quin hexàgon hi ha el lladre. Aquest hexàgon no produeix. Quan surt un 7 o es juga un CAVALLER, la partida queda amb el lladre pendent (camp lladrePendent) fins que el jugador actiu el mou.

- **CartaDesenvolupament** (part avançada): CAVALLER, MONOPOLI i PUNT_VICTORIA. Mentre són a la pila pertanyen a la partida i no tenen propietari; quan un jugador en compra una, passa a la seva mà i es guarda la ronda de compra (rondaCompra).
    - CAVALLER: el jugador mou el lladre a un altre hexàgon i roba 1 recurs aleatori a un altre jugador que tingui un assentament en aquell hexàgon. No provoca descart. Compta per a l'exèrcit més gran.
    - MONOPOLI: el jugador tria un tipus de recurs i tots els altres jugadors li donen tots els recursos d'aquest tipus que tinguin.
    - PUNT_VICTORIA: no es juga; val 1 punt de victòria mentre el jugador la tingui a la mà.

En aquest diagrama UML, la classe Partida representa una partida en curs: té l'estat (PREPARACIO, EN_CURS, FINALITZADA), la ronda, l'última tirada de daus, el jugador actiu, els punts necessaris per guanyar, si el jugador actiu ja ha tirat els daus (dausTirats), si hi ha un moviment de lladre pendent (lladrePendent) i el guanyador. Cada Partida té el seu propi tauler: els Hexagon, Vertex i Aresta es creen en iniciar la partida a partir dels fitxers de topologia. Un Hexagon té 6 Vertex; aquesta relació es guarda a l'entitat HexagonVertex, que també guarda la posició (0-5), i una Aresta uneix exactament dos Vertex. El Jugador pertany a una única Partida, té un inventari de RecursJugador i és el propietari de les seves Construccio. Poblat i Ciutat (a través d'Assentament) apunten a un Vertex i Carretera apunta a una Aresta; un vèrtex o una aresta només pot tenir una construcció.



* Totes les taules han de tenir clau primària. No es fan servir claus compostes: totes les entitats tenen un id autogenerat, també RecursJugador i HexagonVertex.

Costos de construcció

- Carretera: 1 FUSTA + 1 MAÓ
- Poblat: 1 FUSTA + 1 MAÓ + 1 LLANA + 1 BLAT
- Ciutat: 2 BLAT + 3 MINERAL
- Carta de desenvolupament (avançat): 1 BLAT + 1 LLANA + 1 MINERAL
- Comerç amb la banca: 4 recursos iguals → 1 recurs qualsevol. La banca té recursos il·limitats

Fases del Joc

Inici:

- Es carrega la topologia del tauler (hexàgons, vèrtexs i arestes) des dels fitxers CSV
- S'assignen aleatòriament els terrenys als 19 hexàgons: 4 BOSC, 4 PASTURES, 4 CAMPS, 3 TURONS, 3 MUNTANYES i 1 DESERT
- S'assignen aleatòriament els números als 18 hexàgons que no són desert: 2, 3, 3, 4, 4, 5, 5, 6, 6, 8, 8, 9, 9, 10, 10, 11, 11, 12
- El lladre comença al desert
- Es crea l'ordre de torn aleatòriament i tots els jugadors comencen amb 0 recursos

Fase de PREPARACIO:

En ordre de serp (ronda 1: jugadors 1, 2, 3, 4; ronda 2: 4, 3, 2, 1), a cada torn el jugador col·loca sense pagar exactament 1 poblat i, després, 1 carretera que toqui aquest poblat (un poblat propi que encara no tingui cap carretera pròpia adjacent). Es fan dues rondes (dos poblats i dues carreteres per jugador). Quan un jugador col·loca el seu segon poblat, rep immediatament 1 recurs per cada hexàgon (no desert) que el toca. Quan tots han acabat, la partida passa a EN_CURS, ronda 1, amb el jugador d'ordreTorn 1 com a actiu

Torn:

Cada jugador tindrà el torn de forma rotatòria segons l'ordre de torn. La ronda s'incrementa cada cop que el torn torna al jugador d'ordreTorn 1

- Tirar daus: es tiren 2 daus de 6 cares. Tots els hexàgons de la partida amb aquest número (i on no hi hagi el lladre) produeixen: 1 recurs per cada poblat adjacent i 2 per cada ciutat adjacent, que van al propietari. Si surt 7, no hi ha producció (a la part avançada actua el lladre). És obligatori tirar els daus abans de fer qualsevol altra acció del torn
- Construir i comerciar: el jugador pot fer tantes accions com vulgui mentre tingui recursos:

  - Construir una carretera
  - Construir un poblat
  - Millorar un poblat a ciutat
  - Comerciar amb la banca (4:1)
  - Comprar o jugar una carta de desenvolupament (avançat)
- Passar torn: el jugador actiu passa a ser el següent

Regles de col·locació:

- Carretera: l'aresta ha d'estar lliure i ha de tocar un vèrtex amb un poblat/ciutat del jugador o compartir un vèrtex amb una aresta on hi hagi una carretera del jugador. Simplificació: un poblat rival en aquest vèrtex no talla la connexió
- Poblat: el vèrtex ha d'estar lliure, cap vèrtex adjacent pot tenir cap construcció (regla de la distància) i ha d'estar connectat a una carretera del jugador, és a dir, alguna aresta que tingui aquest vèrtex com a vertexA o vertexB té una carretera del jugador (excepte a la fase PREPARACIO)
- Ciutat: només sobre un poblat del mateix jugador; el poblat desapareix i el substitueix la ciutat

Límits per jugador: 15 carreteres, 5 poblats i 4 ciutats. Els límits compten les construccions que hi ha al tauler: un poblat millorat a ciutat deixa de comptar com a poblat

Final de la partida:

- Punts de victòria: cada poblat 1 punt, cada ciutat 2 punts, cada carta PUNT_VICTORIA 1 punt (avançat), exèrcit més gran 2 punts (avançat)
- Quan un jugador arriba als punts per guanyar (10 per defecte, configurable a la partida), la partida passa a FINALITZADA i es guarda el guanyador. ComprovarVictoria es crida després de cada acció que pot donar punts (construir un poblat o una ciutat, comprar una carta, jugar un CAVALLER)

Requisits de la pràctica

Part Hibernate

Crear la Base de Dades amb Hibernate, amb totes les classes i relacions. Tots els camps han d'estar verificats (nullable, unique, longitud, valors mínims i màxims). L'herència de Construccio ha de ser amb estratègia JOINED. Les relacions de Partida que encara no tenen valor quan es crea la partida (lladre, jugador actiu, guanyador) han de ser nullable. No es poden fer servir claus compostes.

Omplir inicialment la base de dades per iniciar una partida: la topologia es llegeix dels fitxers CSV, no es pot escriure a mà al codi.

Com a mínim dues de les consultes han d'estar definides amb @NamedQuery.

Els mètodes principals han d'implementar-se amb el patró DAO:

- Partida.IniciarPartida(List<String> nomsJugadors) → Inicialitza una nova partida i torna el seu id: Hi ha d'haver entre 2 i 4 jugadors; el color s'assigna segons l'ordre de torn.
  - Crea la partida en estat PREPARACIO i els jugadors amb ordre de torn aleatori
  - Carrega la topologia (hexàgons, vèrtexs, arestes) des dels CSV
  - Assigna terrenys i números aleatòriament i col·loca el lladre al desert
- Partida.MostrarTauler(int idPartida) → Mostra tots els hexàgons de la partida (terreny, número, si hi ha el lladre) i, per cada un, les construccions dels seus vèrtexs amb el propietari, mitjançant una consulta a la BD. Les carreteres no cal mostrar-les.
- Jugador.MostrarRecursos(int idJugador) → Mostra l'inventari de recursos d'un jugador mitjançant una consulta a la BD.
- Partida.MostrarPartida(int idPartida) → Mostra la situació de la partida en un moment donat extraient les dades de la BD. És a dir:
  - Estat, ronda, jugador actiu i última tirada
  - Per cada jugador: punts de victòria, nombre de poblats, ciutats i carreteres (els punts inclouen les cartes PUNT_VICTORIA)
  - Nombre total de recursos de cada jugador (només el total: els tipus són secrets)
- Partida.TirarDaus(int idPartida) → Tira els daus i executa la producció:
  - Comprova que el jugador actiu no hagi tirat ja aquest torn (camp dausTirats)
  - Genera la tirada (2 daus de 6) i la guarda a la partida i marca dausTirats
  - Busca els hexàgons de la partida amb aquest número que no tinguin el lladre
  - Per cada poblat adjacent suma 1 recurs al propietari i per cada ciutat en suma 2
  - Si surt 7 no hi ha producció. A la part avançada, tots els jugadors amb més de 7 recursos en descarten la meitat (arrodonint cap avall), triats aleatòriament, i la partida queda amb el lladre pendent
  - Torna un resum de què ha rebut cada jugador
- Jugador.ConstruirCarretera(int idJugador, int idAresta) → Construeix una carretera:
  - Comprova que sigui el torn del jugador i que l'aresta sigui de la seva partida i estigui lliure

    - Comprova la connectivitat (vegeu regles de col·locació) (en PREPARACIO, ha de tocar el poblat que s'acaba de col·locar)
    - Comprova els recursos (excepte en PREPARACIO) i el límit de 15 carreteres
    - Descompta els recursos (excepte en PREPARACIO) i crea la carretera
- Jugador.ConstruirPoblat(int idJugador, int idVertex) → Construeix un poblat:
    - Comprova torn, vèrtex lliure, regla de la distància i connexió amb una carretera pròpia (la connexió no es comprova en PREPARACIO)
    - Comprova els recursos (excepte en PREPARACIO) i el límit de 5 poblats
    - Descompta els recursos (excepte en PREPARACIO) i crea el poblat. Si és el segon poblat de la PREPARACIO, reparteix els recursos inicials
- Jugador.MillorarACiutat(int idJugador, int idVertex) → Substitueix un poblat propi per una ciutat:
    - Comprova torn, que al vèrtex hi hagi un poblat del jugador, els recursos i el límit de 4 ciutats
    - Descompta els recursos, elimina el poblat i crea la ciutat
- Jugador.ComerciarBanca(int idJugador, TipusRecurs dona, TipusRecurs rep) → Canvia 4 recursos del tipus dona per 1 del tipus rep. Comprova torn, que en tingui 4 i que dona i rep siguin diferents.
- int Jugador.CalcularPunts(int idJugador) → Calcula els punts de victòria del jugador amb una consulta a la BD (poblats i ciutats; a la part avançada també cartes PUNT_VICTORIA i exèrcit més gran).
- Jugador.PassarTorn(int idJugador) → Gestiona el final del torn d'un jugador:
    - Comprova que sigui el jugador actiu i que hagi tirat els daus (si la partida és EN_CURS). En PREPARACIO, comprova que el jugador hagi col·locat el poblat i la carretera d'aquest torn (en té tants com el número de ronda). Amb el lladre pendent no es pot passar torn
    - Actualitza el jugador actiu i, si cal, la ronda (en PREPARACIO, seguint l'ordre de serp) i posa dausTirats a fals
    - Si la partida és en PREPARACIO i tots els jugadors tenen 2 poblats i 2 carreteres, passa la partida a EN_CURS, ronda 1, amb el jugador d'ordreTorn 1 com a actiu
- Partida.ComprovarVictoria(int idPartida) → Comprova les condicions de victòria:
    - Calcula els punts de cada jugador
    - Si algun jugador arriba als punts per guanyar, la partida passa a FINALITZADA i es guarda el guanyador (camp guanyador)
- Jugador.MoureLladre(int idJugador, int idHexagon, Integer idJugadorVictima) → Mou el lladre quan està pendent (després d'un 7 o d'un CAVALLER):
    - Comprova que sigui el jugador actiu i que la partida tingui el lladre pendent

  - Mou el lladre a l'hexàgon indicat (ha de ser diferent de l'actual)
  - Roba 1 recurs aleatori a la víctima, que ha de ser un altre jugador amb un assentament en aquell hexàgon. Si cap jugador compleix la condició o cap no té recursos, idJugadorVictima és null i no es roba res
  - Posa lladrePendent a fals
- Jugador.ComprarCarta(int idJugador) → Compra una carta de desenvolupament: comprova torn i recursos, descompta i mou una carta de la pila de la partida a la mà del jugador. La carta s'agafa aleatòriament de la pila i es guarda la ronda de compra. Si la pila és buida, l'acció falla.
- Jugador.JugarCarta(int idJugador, int idCarta, TipusRecurs recurs) → Juga una carta de la mà (no es pot jugar a la mateixa ronda en què s'ha comprat, és a dir, rondaCompra ha de ser menor que la ronda actual). La carta queda marcada com a usada. El paràmetre recurs només s'usa amb MONOPOLI:
  - CAVALLER: incrementa els cavallers jugats del jugador i deixa el lladre pendent; després el jugador crida MoureLladre (sense descart)
  - MONOPOLI: tots els altres jugadors donen al jugador tots els recursos del tipus indicat
  - PUNT_VICTORIA: no es juga, compta directament a CalcularPunts
- Partida.ExercitMesGran(int idPartida) → Torna el jugador amb l'exèrcit més gran: el que ha jugat més cavallers, sempre que en tingui almenys 3 i cap altre n'hagi jugat els mateixos. Aquest jugador té 2 punts extra. Simplificació respecte del joc real, on el conserva qui l'ha aconseguit primer fins que algú el supera.
- List<Vertex> Jugador.VertexsDisponibles(int idJugador) → Torna els vèrtexs on el jugador pot construir un poblat ara mateix (lliures, que compleixin la regla de la distància i estiguin connectats a una carretera pròpia). S'ha de resoldre amb consultes JPQL, no recorrent tots els vèrtexs en Java. En PREPARACIO no cal la connexió amb una carretera.

Part Spring

Instal·lar Spring Tools Suite i crear un projecte Spring Boot Project amb les següents característiques

- Maven (No Gradle)
- Llibreries:
  - Spring Data JPA
  - MySQL Driver
  - Spring Web

Fer els següents endpoints amb Spring (recorda de crear els Repositories i Controllers):

- GET /partida/{idPartida}/tauler → Mostra els hexàgons de la partida amb terreny, número, lladre i construccions adjacents.
  - Fes servir PathVariable
  - Torna un ArrayJSON
- POST /partida → Inicialitza una nova partida.
  - Rep amb RequestBody un JSON amb la llista de noms dels jugadors i, opcionalment, els punts per guanyar
  - Torna un JSON amb la partida, incloent-hi els jugadors i els hexàgons amb el seu terreny i número
- POST /partida/{idPartida}/daus → Tira els daus i executa la producció.
  - Fes servir PathVariable
  - Torna un JSON amb la tirada i els recursos que ha rebut cada jugador
- GET /jugador/{idJugador}/recursos → Mostra el jugador amb el seu inventari.
  - Fes servir PathVariable
  - Torna el jugador (amb els seus recursos) dins d'un ResponseEntity; si el jugador no existeix, torna un 404
- POST /jugador/{idJugador}/construir → Construeix una carretera, un poblat o una ciutat.
  - Fes servir PathVariable per al jugador i RequestParam per a tipus (CARRETERA, POBLAT o CIUTAT) i posicio (id de l'aresta o del vèrtex)
  - Torna una String amb el que passa (la String ha de ser diferent per construït, no és el teu torn, posició ocupada, regla de la distància, no connectat, sense recursos, límit assolit, partida no en curs, daus no tirats, no hi ha cap poblat teu per millorar, posició d'una altra partida)
- POST /jugador/{idJugador}/passar → Passa el torn (Jugador.PassarTorn).
  - Fes servir PathVariable
  - Torna una String amb el resultat (torn passat o el motiu pel qual no es pot passar)

Entrega

Hi haurà 2 entregues grupals en les quals ha d'apareixer el nom de tots els membres del grup:

- Entrega Hibernate:
  - Projecte JAVA amb la resolució
  - Fitxer readme.txt amb comentaris
  - Document de proves
- Entrega Spring:
  - Projecte JAVA amb el webservice

- Fitxer readme.txt amb comentaris
- Document de proves

Avaluació

Hibernate

- Part base 5,5p
  - La part base no requereix les cartes de desenvolupament ni el lladre (amb un 7 simplement no hi ha producció)
- Part avançada: Les funcions que estan en vermell. 3,5p
- Document de proves (0,5 Punts)
- Netedat del codi, comentaris i ús correcte de les anotacions JPA (0,5 Punts)

Spring

- Part base 4p
- Part avançada: Les funcions que estan en vermell. 5p
  - 3p construïr i 2p passar
- Document de proves (0,5 Punts)
- Netedat del codi, comentaris i ús correcte de les anotacions JPA (0,5 Punts)

Consideracions

Fitxers de topologia del tauler (s'entreguen amb l'enunciat):

- hexagons.csv: id de l'hexàgon i coordenades axials q, r (només informatives)
- vertexs.csv: id del vèrtex i coordenades x, y (només per dibuixar)
- hexagon_vertex.csv: quin vèrtex pertany a quin hexàgon i en quina posició (0-5) Es guarda a l'entitat HexagonVertex (id, hexagon, vertex, posicio).
- arestes.csv: cada aresta uneix dos vèrtexs. Dos vèrtexs són adjacents si existeix una aresta entre ells

El camp id dels CSV es guarda com a codi. Com que cada partida crea el seu propi tauler, l'id de la BD no coincideix amb el codi a partir de la segona partida: els mètodes i els endpoints treballen sempre amb l'id de la BD.

Com que cada jugador pertany a una única partida, per qualsevol acció (construir, comerciar, etc.) la partida s'obté sempre a partir del jugador. TirarDaus rep l'idPartida i actua sobre el jugador actiu.

Totes les accions de jugador han de comprovar que la partida estigui EN_CURS (o PREPARACIO per als dos primers poblats i carreteres) i que el jugador sigui l'actiu. Si no, l'acció ha de fallar amb un missatge clar. Durant EN_CURS, a més, cal haver tirat els daus i no tenir el lladre pendent (excepte TirarDaus i MoureLladre). Una partida FINALITZADA no admet cap acció.

Com funciona la producció:

Amb una tirada de 8, cerca els hexàgons de la partida amb número 8 sense lladre. Per cada un, mira els seus 6 vèrtexs: si hi ha un poblat, el propietari rep 1 recurs del terreny; si hi ha una ciutat, en rep 2. Un mateix jugador pot cobrar de diversos hexàgons en una mateixa tirada

Com funciona la regla de la distància:

Si el jugador A té un poblat al vèrtex 10 i els vèrtexs 9 i 11 són adjacents al 10, ningú (ni el mateix A) pot construir un poblat al 9 ni a l'11

Com funciona la connectivitat:

Una carretera nova ha de tocar (per un dels dos vèrtexs de la seva aresta) una construcció o una carretera del mateix jugador. Un poblat nou ha d'estar en un vèrtex on arribi una carretera del mateix jugador

Altres consideracions:

- Cartes de desenvolupament (avançat): la pila té 14 CAVALLER, 5 PUNT_VICTORIA i 2 MONOPOLI (ComprarCarta n'agafa una a l'atzar). Es creen a IniciarPartida.
- Lladre (avançat): quan surt un 7, TirarDaus ha d'aplicar el descart de la meitat de recursos i després el jugador crida MoureLladre per triar l'hexàgon i la víctima. Amb un CAVALLER no hi ha descart.
- Simplificacions respecte del joc real: no hi ha carretera més llarga, ports, comerç entre jugadors ni cartes de progrés, i la banca té recursos il·limitats.
- Serialització JSON (Spring): les relacions bidireccionals provoquen recursió infinita en convertir les entitats a JSON. Cal fer servir @JsonIgnore / @JsonManagedReference i @JsonBackReference, o bé DTOs.